

Manipulação *de arquivos IV*

Nem todo arquivo é texto: imagens, PDFs, planilhas e arquivos compactados são **binários**, sequências de bytes. Nesta última aula da série: o modo `"b"`, a diferença entre `str` e `bytes`, descobrir o tipo real de um arquivo pelos primeiros bytes, copiar arquivos grandes em blocos, pular para uma posição com `seek`, usar arquivos temporários, criar e ler `.zip` e gravar sem risco de corromper o arquivo se o programa cair no meio.

`"rb"` · `"wb"`

`str` × `bytes`

Assinatura do arquivo

Cópia em blocos

`seek` · `tell`

`tempfile`

`zipfile`

`os.replace`

Formação Claude Code 2026 · Baseado em docs.python.org (Built-in Types: bytes; io: seek, tell; tempfile; zipfile; os.replace; Unicode HOWTO), consultados em outubro de 2026 · Exemplos testados com Python 3.14 em `exemplos/` · Saídas reais

str e bytes: *texto e números*

binario.py

```
texto = "Olá"
dados = texto.encode("utf-8")          # str -> bytes
print(dados, len(texto), len(dados))

with open("saudacao.bin", "wb") as f:   # "b": modo binário
    f.write(dados + bytes([0, 255]))

with open("saudacao.bin", "rb") as f:
    lido = f.read()
    print(lido)
    print(list(lido))                  # cada byte é um número
    print(lido[:4].decode("utf-8"))    # bytes -> str
```

SAÍDA

```
b'Ol\xc3\xa1' 3 4
b'Ol\xc3\xa1\x00\xff'
[79, 108, 195, 161, 0, 255]
Olá
```

- ✓ `str` é texto; `bytes` é uma sequência de números de 0 a 255
- ✓ `encode` transforma texto em bytes; `decode` faz o contrário (os dois com um encoding)
- ✓ "Olá" tem 3 caracteres, mas 4 bytes em UTF-8: o "á" ocupa dois
- ✓ Com `"wb"` e `"rb"`, não há `encoding`: o arquivo é gravado e lido byte a byte

O tipo real *está nos primeiros bytes*

assinatura.py

```
from pathlib import Path

ASSINATURAS = {
    b"\x89PNG\r\n\x1a\n": "imagem PNG",
    b"%PDF": "documento PDF",
    b"PK\x03\x04": "arquivo ZIP (ou .docx, .xlsx)",
}

Path("foto.png").write_bytes(b"\x89PNG\r\n\x1a\n" + b"\0" * 20)
Path("contrato.pdf").write_bytes(b"%PDF-1.7\n...")
Path("falso.pdf").write_bytes(b"MZ\x90\x00")  # extensão mente

def tipo_real(caminho):
    inicio = Path(caminho).read_bytes()[:8]
    for magica, nome in ASSINATURAS.items():
        if inicio.startswith(magica):
            return nome
    return "desconhecido"

for nome in ["foto.png", "contrato.pdf", "falso.pdf"]:
    print(f"{nome:13} -> {tipo_real(nome)}")
```

SAÍDA

```
foto.png      -> imagem PNG
contrato.pdf   -> documento PDF
falso.pdf      -> desconhecido
```

- ✓ Muitos formatos começam com uma sequência fixa de bytes (o "número mágico")
- ✓ A extensão é só parte do nome: `falso.pdf` não começa com `%PDF`
- ✓ `.docx` e `.xlsx` são, por dentro, arquivos ZIP

Em uploads, confira o conteúdo, não só a extensão. Os arquivos do exemplo são de mentira: só o começo imita o formato.

Copiar *em blocos*

blocos.py

```
from functools import partial
from pathlib import Path

Path("grande.bin").write_bytes(bytes(range(256)) * 400)

blocos = 0
with open("grande.bin", "rb") as origem, \
     open("copia.bin", "wb") as destino:
    for bloco in iter(partial(origem.read, 4096), b''):
        destino.write(bloco)
        blocos += 1

print(blocos, "blocos de até 4096 bytes")
iguais = Path("grande.bin").read_bytes() == \
        Path("copia.bin").read_bytes()
print("cópia idêntica:", iguais)
```

SAÍDA

```
25 blocos de até 4096 bytes
cópia idêntica: True
```

- ✓ Arquivos binários não têm linhas: leia em blocos de tamanho fixo com `read(4096)`
- ✓ `iter(função, b'')` repete a leitura até vir vazio (aula 02)
- ✓ Só um bloco fica na memória por vez, seja o arquivo de 100 KB ou de 10 GB
- ✓ 102.400 bytes em blocos de 4.096 = 25 blocos

Para só copiar, `shutil.copy` (aula 17) já faz isso por dentro. O laço serve quando você precisa processar cada bloco.

Pulando para *uma posição*

posicao.py

```
with open("registros.bin", "wb") as f:
    for i in range(5):
        f.write(f"REG{i:02d}".encode()) # registros de 5 bytes

with open("registros.bin", "rb") as f:
    print(f.read(5), f.tell())          # leu 5: posição 5
    f.seek(3 * 5)                       # pula para o 4º
    print(f.read(5), f.tell())
    f.seek(-5, 2)                       # 5 bytes antes do fim
    print(f.read())
```

SAÍDA

```
b'REG00' 5
b'REG03' 20
b'REG04'
```

CHAMADA

FAZ

`f.tell()`

diz a posição atual, em bytes

`f.seek(n)`

vai para o byte `n` a partir do início

`f.seek(n, 1)`

anda `n` a partir de onde está

`f.seek(-n, 2)`

`n` bytes antes do fim

Com registros de tamanho fixo, dá para ler o 4º sem passar pelos anteriores: posição = índice × tamanho.

Arquivos que *se apagam sozinhos*

temporarios.py

```
import tempfile
from pathlib import Path

with tempfile.TemporaryDirectory() as pasta:
    rascunho = Path(pasta) / "rascunho.txt"
    rascunho.write_text("rascunho", encoding="utf-8")
    print("existe durante o bloco:", rascunho.exists())

print("existe depois do bloco:", rascunho.exists())
```

SAÍDA

```
existe durante o bloco: True
existe depois do bloco: False
```

- ✓ `TemporaryDirectory()` cria uma pasta com nome único na área temporária do sistema
- ✓ No fim do `with`, a pasta e tudo dentro dela são apagados
- ✓ Útil para etapas intermediárias, testes e arquivos que não devem ficar para trás
- ✓ Para um arquivo só, existe o `tempfile.NamedTemporaryFile`

Criando e lendo *um .zip*

compactar.py

```
import zipfile
from pathlib import Path

Path("leia_me.txt").write_text("Estoque\n" * 100, "utf-8")
Path("dados.csv").write_text("item,qtd\ncaneta,10\n", "utf-8")

with zipfile.ZipFile("pacote.zip", "w",
                    compression=zipfile.ZIP_DEFLATED) as z:
    z.write("leia_me.txt")
    z.write("dados.csv")

with zipfile.ZipFile("pacote.zip") as z:
    for info in z.infolist():
        print(info.filename, info.file_size, "->",
              info.compress_size)
    print(z.read("dados.csv").decode("utf-8"), end="")
```

SAÍDA

```
leia_me.txt 800 -> 18
dados.csv 19 -> 21
item,qtd
caneta,10
```

- ✓ `ZIP_DEFLATED` compacta; sem ele, o zip só junta os arquivos
- ✓ `infolist()` mostra o tamanho original e o compactado de cada um
- ✓ `read()` devolve `bytes`: `decode` transforma em texto
- ✓ Texto repetitivo encolhe muito (800 → 18 bytes); arquivo pequeno pode até crescer (19 → 21)

Nunca deixar *meio arquivo*

gravacao_segura.py

```
import json
import os
from pathlib import Path

def salvar_json_seguro(dados, caminho):
    caminho = Path(caminho)
    temporario = caminho.with_suffix(".tmp")
    with open(temporario, "w", encoding="utf-8") as f:
        json.dump(dados, f, ensure_ascii=False)
    os.replace(temporario, caminho)    # troca de uma vez só

salvar_json_seguro({"saldo": 150}, "conta.json")
salvar_json_seguro({"saldo": 90}, "conta.json")
print(Path("conta.json").read_text(encoding="utf-8"))
print(Path("conta.tmp").exists())
```

SAÍDA

```
{"saldo": 90}
False
```

GRAVA

conta.tmp



TERMINA

arquivo
completo



OS.REPLACE

vira conta.json

- ✓ Gravando direto com `"w"`, uma queda de energia no meio deixa o arquivo pela metade
- ✓ Aqui, o arquivo antigo só é trocado quando o novo já está completo
- ✓ `os.replace` faz a troca de uma vez: quem lê vê o antigo ou o novo, nunca um pedaço

Backup *compactado*

Enunciado: compacte todos os `.txt` de uma pasta num arquivo zip e mostre quanto o backup economizou.

Uma resposta possível

```
import zipfile
from pathlib import Path

pasta = Path("notas")
pasta.mkdir(exist_ok=True)
for n in range(1, 4):
    arquivo = pasta / f"aula{n}.txt"
    arquivo.write_text(f"resumo {n}\n" * 30, "utf-8")

with zipfile.ZipFile("backup_notas.zip", "w",
                    compression=zipfile.ZIP_DEFLATED) as z:
    for arquivo in sorted(pasta.glob("*.txt")):
        z.write(arquivo)

original = sum(a.stat().st_size for a in pasta.glob("*.txt"))
compactado = Path("backup_notas.zip").stat().st_size
print(f"{original} bytes -> {compactado} bytes")
print(zipfile.ZipFile("backup_notas.zip").namelist())
```

SAÍDA

```
810 bytes -> 388 bytes
['notas/aula1.txt', 'notas/aula2.txt', 'notas/aula3.txt']
```

- ✓ `glob` e `stat().st_size` vêm da aula 17
- ✓ O zip guarda o caminho relativo (`notas/aula1.txt`)
- ✓ Desafio: ponha a data no nome do zip com `date.today().isoformat()`

Teste o que *ficou*

1. "Olá".encode("utf-8") devolve:

A) "Olá"

B) b'Ol\xc3\xa1'

C) [79, 108, 97]

D) Erro

B.

2. Para ler uma imagem, abra com:

A) "r"

B) "rb"

C) "w"

D) "a"

B.

3. f.seek(0) faz:

A) Apaga o arquivo

B) Volta ao início

C) Vai para o fim

D) Fecha o arquivo

B.

4. Para não corromper um arquivo se o programa cair durante a gravação:

A) Gravar com "a"

B) Gravar num temporário e usar os.replace

C) Usar readlines()

D) Gravar duas vezes

B.

Bytes, blocos *e gravação segura*

Arquivos binários se abrem com `"rb"` / `"wb"` e trabalham com `bytes`; `encode` e `decode` fazem a ponte com o texto. Os primeiros bytes revelam o tipo real; arquivos grandes se leem em blocos; `seek` pula para uma posição; `tempfile` limpa sozinho; `zipfile` compacta; e gravar num temporário com `os.replace` protege os dados. Neste módulo você foi de iteradores e geradores a regex, validação, funções de alta ordem, decoradores, código assíncrono, metaclasses, tipos e arquivos.

bytes

Assinatura

Blocos

seek

tempfile

zipfile

os.replace

Fim do módulo 38 · Python Intermediário

Fontes: docs.python.org/3/library/stdtypes.html#bytes; docs.python.org/3/howto/unicode.html; docs.python.org/3/library/io.html (seek, tell); docs.python.org/3/library/tempfile.html; docs.python.org/3/library/zipfile.html; docs.python.org/3/library/os.html#os.replace, consultados em outubro de 2026. Exemplos em [exemplos/](#), executados com Python 3.14.4.